M19 GitHub Actions + Claude Code

Méthodologie Advanced

Intégrer Claude Code dans les pipelines CI/CD GitHub

Pattern recommandé : prompt externalisé

Ce pattern sépare la logique de review de la mécanique du workflow. Le fichier YAML orchestre les déclencheurs et permissions ; le fichier `.github/prompts/code-review.md` contient les critères de review. Modifier les critères ne nécessite pas de toucher au workflow.

```
.github/
├── workflows/
│   └── claude-review.yml    # Mécanique CI
└── prompts/
    └── code-review.md       # Critères, protocoles
```

Cette séparation permet d'itérer sur la qualité des reviews sans risquer de casser le pipeline.

Authentification : OAuth vs API key

Méthode	Coût par review	Prérequis
OAuth abonnement	Consomme le quota du plan ; limites applicables	Compte éligible et token OAuth
`ANTHROPIC_API_KEY`	Facturation API au token	Clé API Anthropic

Choisissez l'authentification selon le compte et le volume de travail. Une revue sur abonnement consomme le quota ; aucune méthode ne garantit un coût fixe par revue.

Workflow minimal commenté

```yaml
on:
  pull_request:
    types: [opened, synchronize]
  issue_comment:
    types: [created]

jobs:
  claude-review:
    if: |
      github.event_name == 'pull_request' ||
      (github.event.issue.pull_request &&
       contains(github.event.comment.body, '/claude-review'))
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: anthropics/claude-code-action@v1
        with:
          claude_code_oauth_token:
${{ secrets.CLAUDE_CODE_OAUTH_TOKEN }}
          trigger_phrase: /claude-review
          prompt: Read .github/prompts/code-review.md and review the PR.
          claude_args: >-
            --model claude-sonnet-5-5
            --tools "Read,Glob,Grep"
```

`fetch-depth: 0` récupère tout l'historique. L'action actuelle accepte `prompt` et `claude_args` ; la restriction CLI des outils garde cet exemple en lecture seule. Voir les entrées officielles de l'action.

Outils autorisés en CI

En CI, limiter Claude aux outils de lecture uniquement. `Read`, `Glob` et `Grep` couvrent tous les besoins d'une review de code. Ajouter `Write` ou `Bash` crée un risque de modification accidentelle du dépôt pendant l'analyse.

Les outils MCP GitHub (`mcp__github__get_pull_request_diff`, `mcp__github__submit_pending_pull_request_review`) permettent à Claude de poster des commentaires inline sans accès en écriture direct au dépôt.

Anti-hallucination protocol

Les reviews automatisées souffrent d'un problème récurrent : Claude invente des numéros de ligne ou rapporte des issues qu'il n'a pas vérifiées. Le protocol de mitigation consiste à demander explicitement dans le prompt de vérification avant toute assertion.

Formulation dans `code-review.md` :

```
Before reporting any issue, verify it with Read or Grep.
Never cite a line number you have not confirmed.
Structure output: MUST FIX / SHOULD FIX / CAN SKIP.
```

Cette instruction réduit les faux positifs sans complexifier le workflow.

Trigger on-demand : `/claude-review`

Le `issue_comment` trigger permet à n'importe quel membre de l'équipe de déclencher une review à la demande en tapant `/claude-review` dans un commentaire de PR. Utile pour les PRs complexes où une review automatique à l'ouverture n'aurait pas eu assez de contexte.

La condition `github.event.issue.pull_request != null` filtre les commentaires d'issues ordinaires pour n'activer l'agent que sur les PRs.

Gestion des échecs

Prévoir un job de fallback qui poste un commentaire en cas d'échec du job Claude. Sans fallback, une PR peut rester silencieuse pendant des heures si l'agent plante, sans que l'équipe le sache.

```yaml
- name: Handle review failure
  if: failure()
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: '⚠️ Claude review failed — human reviewer needed.'
      });
```